RAAD

Security

DVWA CORP

Security Assessment Findings Report

Business Confidential

Date: May 6th, 2025

Project: DC-005

Version 1.0

Copyright © RAAD Security (raad-security.com)

Table of Contents

Table of Contents 2
Confidentiality Statement 4
Disclaimer 4
Contact Information 4
Assessment Overview 5
Assessment Components 6
Web Application Penetration Test 6
Finding Severity Ratings 7
Risk Factors 7
Likelihood 7
Impact 7
Scope 8
Scope Exclusions 8
Client Allowances 8
Executive Summary 9
Scoping and Time Limitations 9
Testing Summary 9
Key Strengths and Weaknesses 11
Vulnerability Summary 12
Web Application Penetration Test Findings 12
Technical Findings 14
Web Application Penetration Test Findings 14
Finding WAPT-001: File Upload (Critical) 14
Finding WAPT-002: Weak Authentication – Brute Force Vulnerability (Critical) 17
Finding WAPT-003: Remote Code Execution (RCE) via Command Injection (Critical) 19

Copyright © RAAD Security (raad-security.com)

Finding WAPT-004: SQL Injection (Critical) 22
Finding WAPT-005: Outdated / Unsupported Software Stack – Apache, PHP, mod_dav, Ubuntu (Critical) 27
Finding WAPT-006: Cross-Site Scripting (High) 29
Finding WAPT-007: File Inclusion (LFI/RFI) Due to Improper Input Validation (High) 32
Finding WAPT-008: Insufficient CSRF Protection (Medium) 34
Additional Scans and Reports 37

Copyright © RAAD Security (raad-security.com)

Confidentiality Statement

This document is the exclusive property of DVWA Corp and RAAD Security (RAADSEC). This document contains proprietary and confidential information. Duplication, redistribution, or use, in whole or in part, in any form, requires consent of both DVWA Corp and RAADSEC.

DVWA Corp may share this document with auditors under non-disclosure agreements to demonstrate penetration test requirement compliance.

Disclaimer

A penetration test is considered a snapshot in time. The findings and recommendations reflect the information gathered during the assessment and not any changes or modifications made outside of that period.

Time-limited engagements do not allow for a full evaluation of all security controls. RAADSEC prioritized the assessment to identify the weakest security controls an attacker would exploit. RAADSEC recommends conducting similar assessments on an annual basis by internal or third-party assessors to ensure the continued success of the controls.

Contact Information

Name	Title	Contact Information
DVWA Corp		
Walter Wolf	Global Information Security Manager	Email: walter@dvwacorp.com
RAAD Security		
Mohamed Eltohamy	Lead Penetration Tester	Email: mohamed@raad-security.com

Copyright © RAAD Security (raad-security.com)

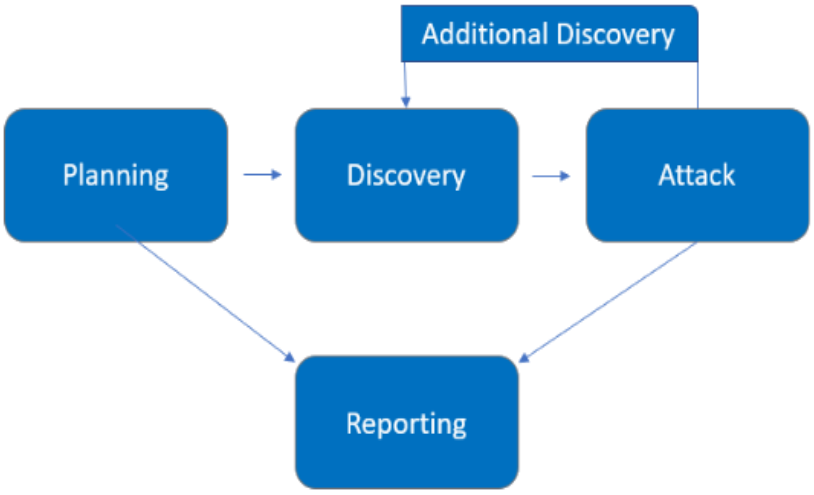
Assessment Overview

From April 25nd, 2025 to May 5th, 2025, DVWA Corp engaged RAADSEC to evaluate the security

posture of its infrastructure compared to current industry best practices that included an internal network penetration test. All testing performed is based on the NIST SP 800-115 Technical Guide to Information Security Testing and Assessment, OWASP Testing Guide (v4), and customized testing frameworks.

Phases of penetration testing activities include the following:

- Planning – Customer goals are gathered and rules of engagement obtained.
- Discovery – Perform scanning and enumeration to identify potential vulnerabilities, weak areas, and exploits.
- Attack – Confirm potential vulnerabilities through exploitation and perform additional discovery upon new access.
- Reporting – Document all found vulnerabilities and exploits, failed attempts, and company strengths and weaknesses.



Copyright © RAAD Security (raad-security.com)

Assessment Components

Web Application Penetration Testing

A web application penetration test is an in-depth penetration test on both the unauthenticated and authenticated portions of your website. The engineer will test for all the OWASP Top-10 critical security flaws, as well as a variety of other potential vulnerabilities based on security best practice. Activities include:

- Website mapping
- Directory enumeration
- Automated and manual injection testing
- Directory traversal testing
- Malicious file uploads and remote code execution
- Password attack and authentication bypass
- Session attack
- Other testing depending on specific site content and languages.

All testing performed follows the OWASP v4 guidelines and checklist.

The following tools are commonly used during our web application assessments:

- Burp Suite Pro
- Nessus
- Nmap
- Netcat
- SQLmap
- Ffuf
- Manual Review

Copyright © RAAD Security (raad-security.com)

Finding Severity Ratings

The following table defines levels of severity and corresponding CVSS score range that are used throughout the document to assess vulnerability and risk impact.

Severity	CVSS V3 Score Range	Definition
Critical	9.0-10.0	Exploitation is straightforward and usually results in system-level compromise. It is advised to form a plan of action and patch immediately.
High	7.0-8.9	Exploitation is more difficult but could cause elevated privileges and potentially a loss of data or downtime. It is advised to form a plan of action and patch as soon as possible.
Medium	4.0-6.9	Vulnerabilities exist but are not exploitable or require extra steps such as social engineering. It is advised to form a plan of action and patch after high-priority issues have been resolved.
Low	0.1-3.9	Vulnerabilities are non-exploitable but would reduce an organization's attack surface. It is advised to form a plan of action and patch during the next maintenance window.
Informational	N/A	No vulnerability exists. Additional information is provided regarding items noticed during testing, strong controls, and additional documentation.

Risk Factors

Risk is measured by two factors: **Likelihood** and **Impact**:

Likelihood

Likelihood measures the potential of a vulnerability being exploited. Ratings are given based on the difficulty of the attack, the available tools, attacker skill level, and client environment.

Impact

Impact measures the potential vulnerability's effect on operations, including confidentiality, integrity, and availability of client systems and/or data, reputational harm, and financial loss.

Copyright © RAAD Security (raad-security.com)

Scope

Assessment	Details
Web Application Penetration Test	19x.xx.xx.xx

Scope Exclusions

Per client request, TCMS did not perform any of the following attacks during testing:

- Denial of Service (DoS)
- Phishing/Social Engineering

Client Allowances

DVWA Corp provided RAADSEC the following allowances:

- Two user accounts for testing
- Source code access

Copyright © RAAD Security (raad-security.com)

Executive Summary

TCMS evaluated Demo Corp's internal security posture through penetration testing from April 25nd, 2025 to May 5th,2025. The following sections provide a high-level overview of vulnerabilities discovered, successful and unsuccessful attempts, and strengths and weaknesses.

Scoping and Time Limitations

Scoping during the engagement did not permit denial of service or social engineering attacks.

Time limitations were in place for testing. The web application penetration testing (WAPT) was permitted for five (5) business days.

The testing team was provided account access the web application. The RAADSEC team has factored these allowances into overall risk and severity of the findings.

Testing Summary

The web application assessment included vulnerability scanning and active testing on unauthenticated and authenticated user account access. The web application code review included vulnerability scanning and manual review on the Reports Plus source code. Methodology strictly followed both the NIST and OWASP testing guidelines and framework, which allowed for thorough discovery of vulnerabilities.

The assessment of the DVWA Corp Reports Plus application identified multiple security findings, ranging from Medium to Critical severity, which require remediation by the development team. Key findings include:

- SQL Injection (Finding WAPT-004): Enables unauthorized access to sensitive database content.
- File Inclusion via Path Traversal (Finding WAPT-007): Allows access to sensitive system files.
- Stored and Reflected Cross-Site Scripting (XSS) (Finding WAPT-006): Enables execution of malicious scripts in user browsers.
- Command Injection (Finding WAPT-003): Allows execution of arbitrary system commands on the underlying server.

Copyright © RAAD Security (raad-security.com)

- Insufficiently validated file uploads (Finding WAPT-001): Enables Remote Code Execution (RCE). Successful exploitation may allow attackers to gain shell access to the server, compromising confidentiality, integrity, and availability. Authentication and access control weaknesses were also identified:
- The application permits unlimited login attempts and lacks Multi-Factor Authentication (Finding WAPT-002), increasing the risk of brute-force attacks and potential account compromise.
- Insufficient Cross-Site Request Forgery (CSRF) protections (Finding WAPT-008) were found on certain state-changing functionalities, potentially allowing attackers to perform unauthorized actions on behalf of authenticated users via social engineering.

Finally, the application is deployed on outdated and unsupported software components (Finding WAPT-005), significantly increasing exposure to known vulnerabilities and publicly available exploits.

Copyright © RAAD Security (raad-security.com)

Key Strengths and Weaknesses

The following identifies the key strengths identified during the assessment:

1. The web application utilizes HTTPS.
2. The application implements session-based authentication mechanisms.
3. Basic input filtering controls are present at the configured security level.
4. Error handling does not expose detailed stack traces to end users.

The following identifies the key weaknesses identified during the assessment:

1. User-supplied input is not adequately validated or sanitized, resulting in multiple injection-based vulnerabilities including SQL Injection, Cross-Site Scripting (XSS), File Inclusion, and potential Remote Code Execution.
2. Anti-CSRF protections are predictable or improperly validated, allowing unauthorized state-changing requests.
3. Improper Session Invalidation on Logout.
4. Password complexity in insufficient.
5. Lack of rate limiting allows brute forcing of user account.
6. Developer directories and files are exposed to unauthenticated users.
7. Information Disclosure via HTTP Response Headers.
8. Security-related HTTP response headers are missing.
9. Deprecated encryption ciphers are present in the application.
10. File upload functionality lacks strict validation controls.

Copyright © RAAD Security (raad-security.com)

Vulnerability Summary

The following tables illustrates the vulnerabilities found by severity and recommended Recommendations:

Web Application Penetration Test Findings

5	2	1	0	0
Critical	High	Medium	Low	Informational

Finding	Severity	Recommendation
Internal Penetration Test		
WAPT-001: File Upload	Critical	Enforce strict file-type validation (whitelisting) and content inspection.
WAPT-002: Weak Authentication – Brute Force Vulnerability	Critical	Account Lockout: Lock user accounts after a defined number of consecutive failed login attempts (e.g., 5).
WAPT-003: Remote Code Execution (RCE) via Command Injection	Critical	Validate and sanitize all user inputs for type, length, format, and content.
WAPT-004: SQL Injection	Critical	Validate all user inputs for type, length, and format at the point of entry.
Finding WAPT-005: Outdated / Unsupported Software Stack – Apache, PHP, mod_dav, Ubuntu	Critical	Upgrading ensures the application benefits from the latest security patches.
WAPT-006: Cross-Site Scripting	High	Filter input on arrival: Validate and sanitize user input based on expected format and type.
WAPT-007: File Inclusion (LFI/RFI) Due to Improper Input Validation	High	Implement strict allowlisting: Only permit predefined, trusted file names or identifiers. Avoid accepting arbitrary file paths from user input.

Copyright © RAAD Security (raad-security.com)

WAPT-008: Insufficient CSRF Protection	Medium	Implement anti-CSRF tokens**:** Include a unique, unpredictable token in all state-changing requests and validate it server-side

Copyright © RAAD Security (raad-security.com)

Technical Findings

Web Application Penetration Test Findings

Finding WAPT-001: File Upload (Critical)

Description:	The DVWA application allows insufficiently validated file uploads, enabling an attacker to upload a malicious PHP file that can be executed on the server. This vulnerability allows Remote Code Execution (RCE) through arbitrary file upload. An attacker can gain shell access to the server, potentially compromising system confidentiality, integrity, and availability. The issue arises because the application does not properly validate file type, content, or enforce safe storage and execution restrictions.
Risk:	Likelihood: High – Exploitation is straightforward due to lack of proper file validation, allowing attackers to upload and execute arbitrary files. Impact: Critical – Successful exploitation permits Remote Code Execution, potentially compromising system confidentiality, integrity, and availability.
System:	https://19x.xx.xx.xx/dvwa/vulnerabilities/upload/
Tools Used:	Netcat
References:	https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload

Evidence

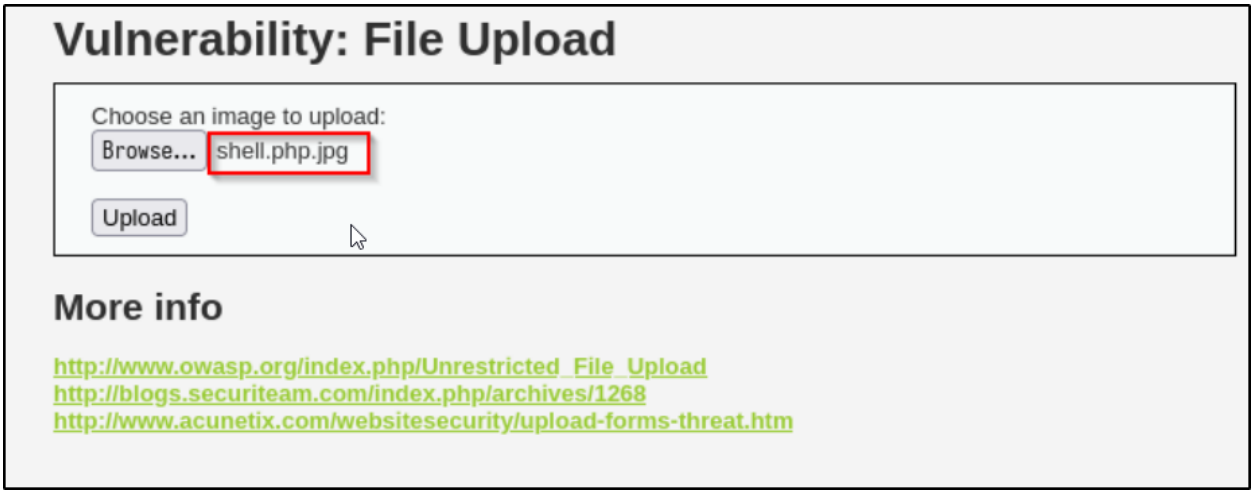


Figure 1: Malicious PHP file selected via the "Choose your image to upload" browse field

Copyright © RAAD Security (raad-security.com)

Figure 2: Malicious PHP file successfully uploaded

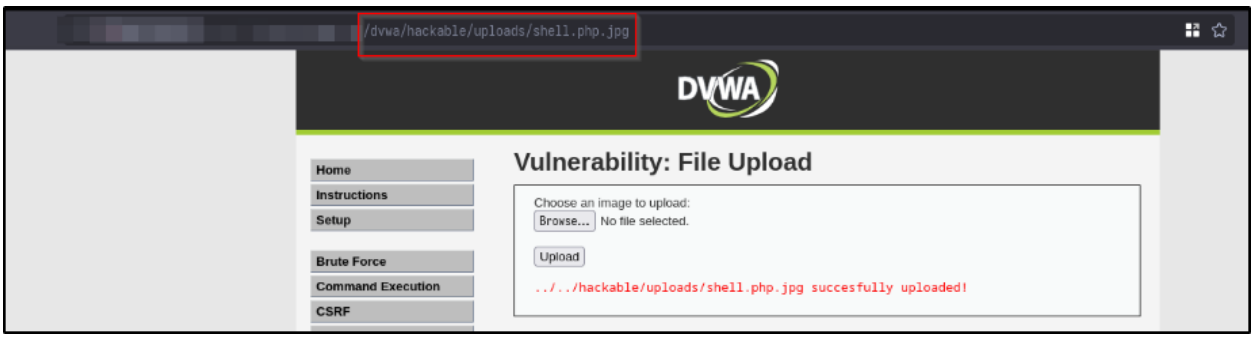


Figure 3: Malicious PHP file executed via the uploaded file URL

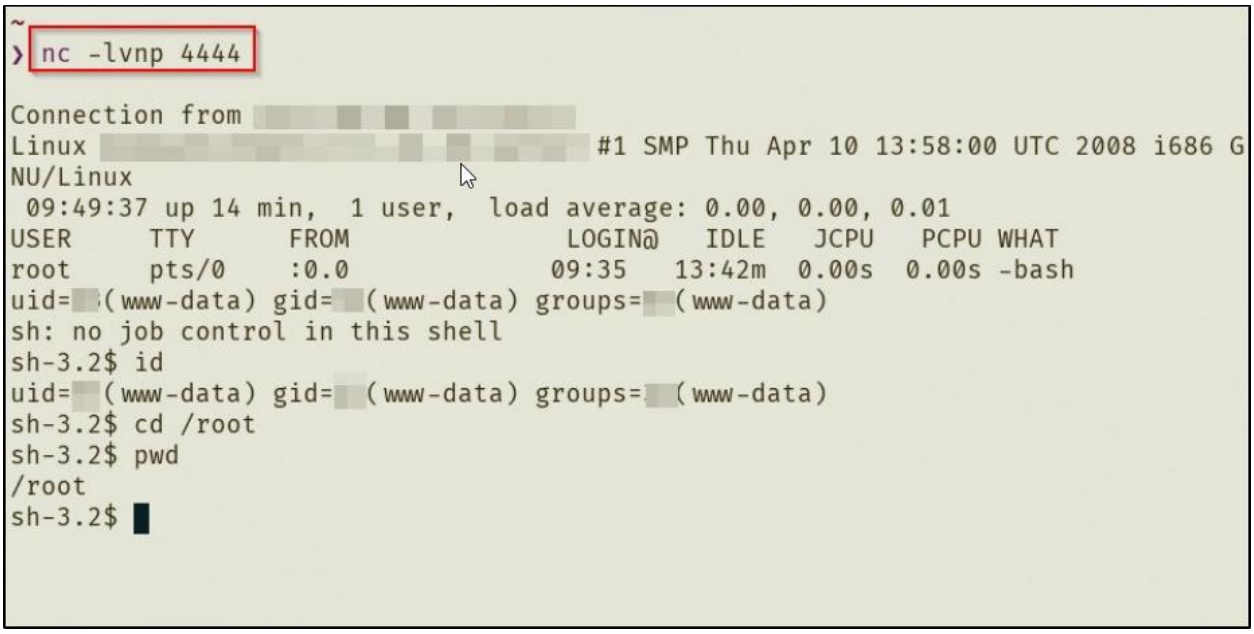


Figure 4: Reverse shell established from the target after accessing the uploaded PHP file

Copyright © RAAD Security (raad-security.com)

Remediation

- Enforce strict file-type validation (whitelisting) and content inspection.
- Prevent execution of uploaded files and store outside the web root.
- Sanitize filenames and generate server-side identifiers.
- Apply size limits and rate limiting for uploads.
- Implement logging, monitoring, and alerting on suspicious activity.
- Harden server and application configurations to minimize attack surface.
- Integrate secure coding practices and regular testing into the SDLC.
- Implementing these measures significantly reduces the risk of arbitrary file upload exploitation and Remote Code Execution (RCE).

Copyright © RAAD Security (raad-security.com)

Finding WAPT-002: Weak Authentication – Brute Force Vulnerability (Critical)

Description:	The DVWA application's login functionality does not implement adequate protections against repeated authentication attempts. During testing, automated brute-force techniques were used to identify valid credentials, demonstrating that the system is susceptible to unauthorized access due to insufficient rate limiting and lack of account lockout mechanisms. This vulnerability allows attackers to gain access to privileged accounts, potentially leading to exposure of sensitive data, unauthorized operations, and lateral movement within the application.
Risk:	Likelihood: High – The attack can be automated using common brute-force tools with minimal technical skill. Impact: High – Successful exploitation results in unauthorized access to privileged accounts, increasing the risk of data exposure and operational compromise.
System:	https://19x.xx.xx.xx/dvwa/vulnerabilities/brute/
Tools Used:	Ffuf
References:	https://owasp.org/www-community/controls/Blocking_Brute_Force_Attacks

---

Evidence

Figure 5: Successful login

Copyright © RAAD Security (raad-security.com)

Remediation

- Account Lockout: Lock user accounts after a defined number of consecutive failed login attempts (e.g., 5).
- Progressive Delays: Introduce exponential backoff or delays after failed attempts to slow automated brute-force tools.
- Strong Passwords & MFA**:** Enforce strong password policies and implement multi-factor authentication to reduce risk of unauthorized access.
- CAPTCHA: Require CAPTCHA validation after 3–5 failed login attempts to ensure the requester is human.
- Rate Limiting: Implement request throttling at the application and/or network layer to prevent high-frequency login attempts.
- Monitoring & Alerts: Monitor login failures and generate alerts for suspicious activity patterns.
- Implementing these measures will significantly reduce the risk of brute-force attacks, protect user accounts, and align the application with secure authentication best practices.

Copyright © RAAD Security (raad-security.com)

Finding WAPT-003: Remote Code Execution (RCE) via Command Injection (Critical)

Description:	The DVWA application allows execution of system commands through insufficiently validated input in the “IP address” field, resulting in a Command Injection vulnerability. This enables an attacker to execute arbitrary OS commands on the web server. Although commands are executed under a non-root web server account (www-data), the attacker is able to access directories typically restricted to root (e.g., /root), exposing sensitive system information without requiring interaction from other
Risk:	Likelihood: High – Exploitation is straightforward due to insufficient input validation, allowing attackers to execute arbitrary system commands via the “IP address” field. Impact: Critical – Successful exploitation allows Remote Code Execution (RCE) under the web server account, potentially exposing sensitive system directories and information, compromising system confidentiality, integrity, and availability.
System:	https://19x.xx.xx.xx/dvwa/vulnerabilities/exec/
Tools Used:	Netcat
References:	https://owasp.org/www-community/attacks/Command_Injection

Evidence

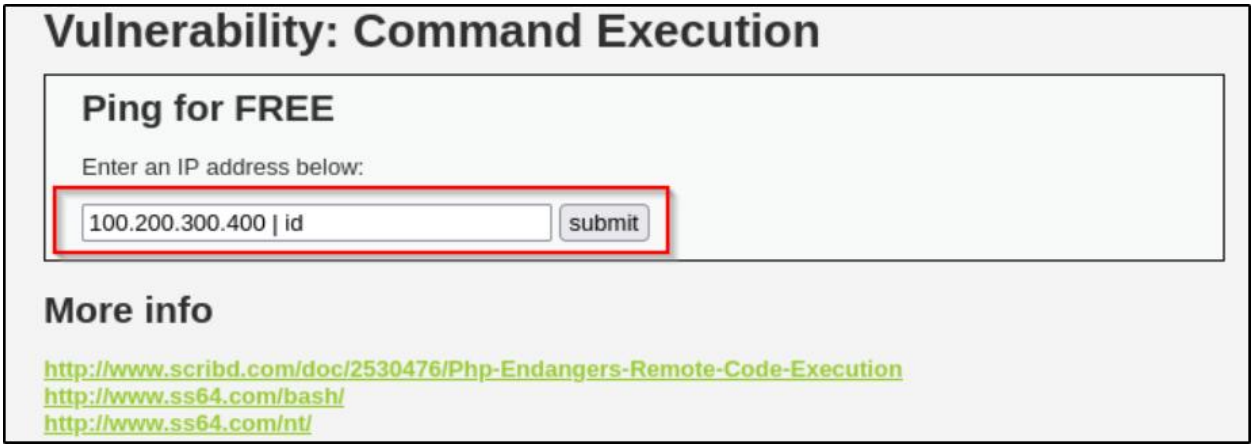


Figure 6: Command payload entered into the "IP address" field

Copyright © RAAD Security (raad-security.com)

Figure 7: Command payload executed after submitting the "Submit" button

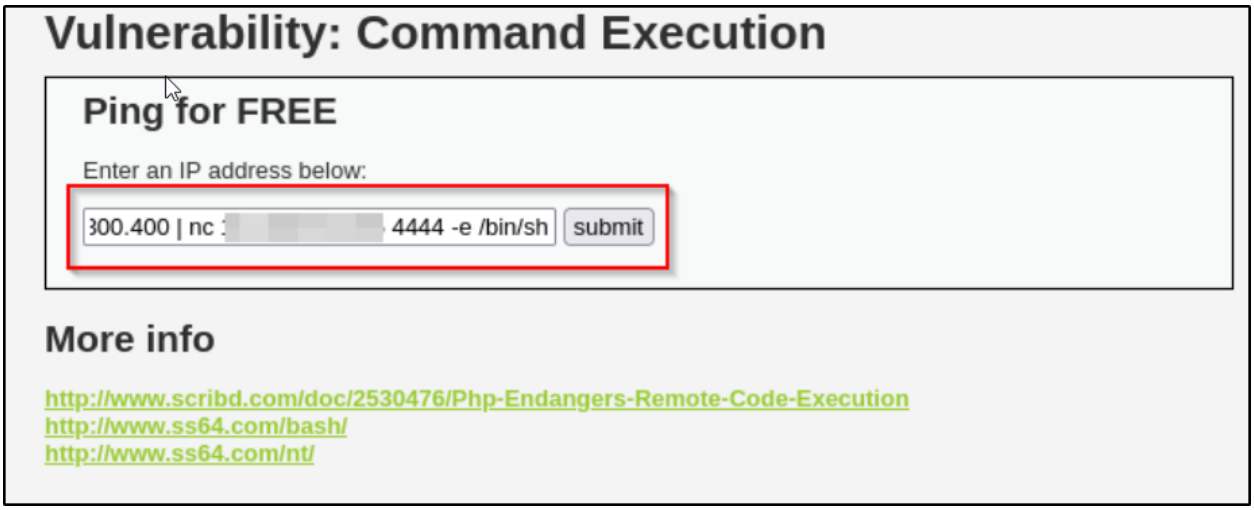


Figure 8: Remote command execution payload entered into the "IP address" field

Copyright © RAAD Security (raad-security.com)

```
~
❯ nc -lvnp 4444

Connection from [redacted]
id
uid=[redacted](www-data) gid=[redacted](www-data) groups=[redacted](www-data)
cd /root
pwd
/root
uname -a
Linux [redacted] #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 G
NU/Linux
lsb_release -a
Distributor ID: Ubuntu
Description:    Ubuntu 8.04
Release:        8.04
Codename:       hardy
groups
www-data
```

Figure 9: Remote command execution successfully triggered after submitting the "Submit" button

Remediation

- Validate and sanitize all user inputs for type, length, format, and content. Reject or escape any characters that could be interpreted as system commands.
- Use safe APIs and avoid direct system calls, replacing any OS command execution with application-level logic.
- Apply the principle of least privilege: ensure web server processes run under restricted accounts and remove unnecessary access to sensitive files and directories.
- Implement monitoring and alerting: log failed or suspicious command execution attempts and set up alerts for potential RCE exploitation.
- Follow secure coding practices to minimize exposure of functionality that could allow system command execution.

Copyright © RAAD Security (raad-security.com)

Finding WAPT-004: SQL Injection (Critical)

Description:	The DVWA application is vulnerable to SQL Injection due to insufficient input validation, allowing attackers to manipulate database queries. Successful exploitation exposes sensitive database information, compromising confidentiality and potentially enabling further attacks on application integrity and availability.
Risk:	Likelihood: High – Exploitation is straightforward and can be automated, requiring minimal technical skill. Impact: Critical – Successful exploitation exposes sensitive database information, potentially affecting confidentiality, integrity, and availability of the system.
System:	https://19x.xx.xx.xx/dvwa/vulnerabilities/sqli/ https://19x.xx.xx.xx/dvwa/vulnerabilities/sqli_blind/
Tools Used:	SQLMap
References:	https://owasp.org/Top10/2021/A03_2021-Injection/index.html https://owasp.org/www-community/attacks/Blind_SQL_Injection

---

Evidence

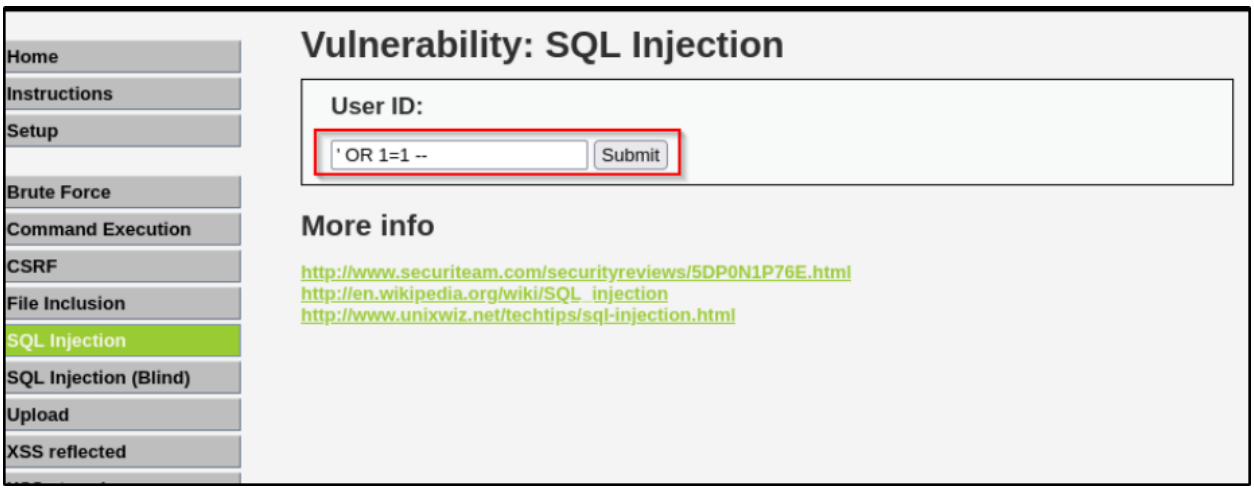


Figure 10: SQL injection payload entered into the "User ID" field

Copyright © RAAD Security (raad-security.com)

Figure 11: Database syntax error triggered by the SQL injection payload

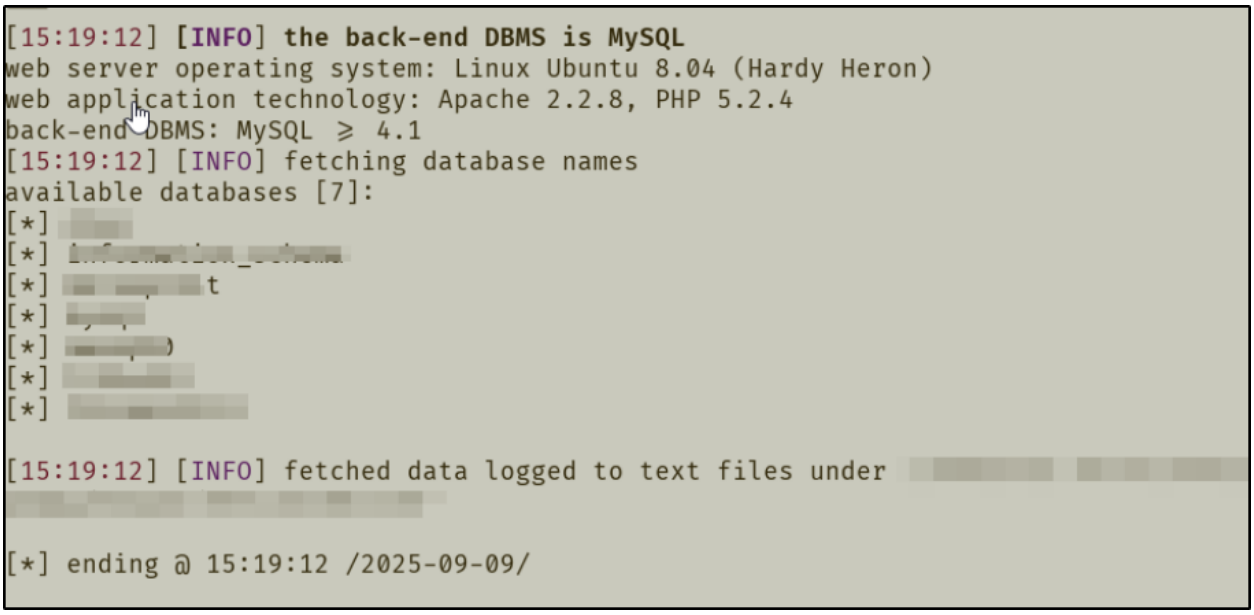


Figure 12: Sqlmap enumeration showing the database name and available databases retrieved

Copyright © RAAD Security (raad-security.com)

```
[14:21:09] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: PHP 5.2.4, Apache 2.2.8
back-end DBMS: MySQL ≥ 4.1
[14:21:09] [INFO] testing if current user is DBA
[14:21:09] [INFO] fetching current user
[14:21:09] [WARNING] reflective value(s) found and filtering out
current user is DBA: True
[14:21:09] [INFO] fetching database users privileges
database management system users privileges:
[*]                    (administrator) [20]:
    privilege: ALTER
    privilege: CREATE
    privilege: CREATE TEMPORARY TABLES
    privilege: DELETE
    privilege: DROP
    privilege: EXECUTE
    privilege: FILE
    privilege: INDEX
    privilege: INSERT
    privilege: LOCK TABLES
    privilege: PROCESS
    privilege: REFERENCES
    privilege: RELOAD
    privilege: REPLICATION CLIENT
    privilege: REPLICATION SLAVE
    privilege: SELECT
```

Figure 13: Sqlmap output showing that the current database user is DBA with critical privileges

```
Database: dvwa
Table: users
[5 entries]
+---------+--------+-------------------------------------+-------------------------------------+-----------+------------+
| user_id | user   | avatar                              | password                            | last_name | first_name |
+---------+--------+-------------------------------------+-------------------------------------+-----------+------------+
| 1       |        |                                     |                                     |           |            |
| 2       |        |                                     |                                     |           |            |
| 3       |        |                                     |                                     |           |            |
| 4       |        |                                     |                                     |           |            |
| 5       |        |                                     |                                     |           |            |
+---------+--------+-------------------------------------+-------------------------------------+-----------+------------+

[14:23:39] [INFO] table 'dvwa.users' dumped to CSV file '
[14:23:39] [INFO] fetched data logged to text files under
```

Figure 14: Partial dump of the 'dvwa.users' table via SQL injection

Copyright © RAAD Security (raad-security.com)

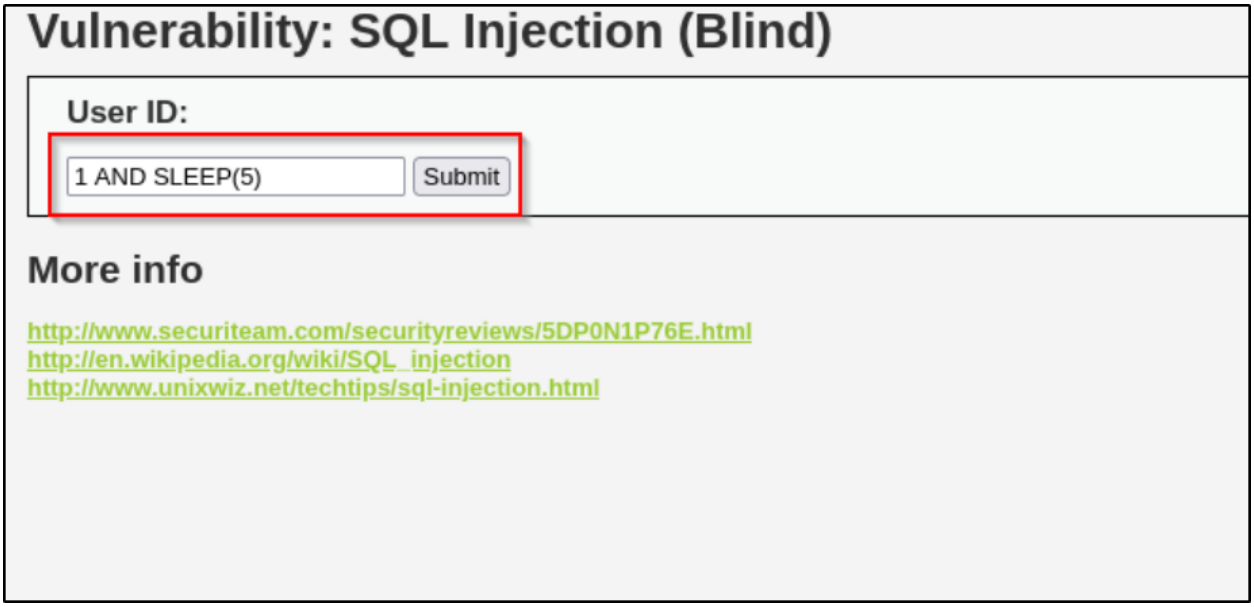


Figure 15: SQL injection payload entered into the "User ID" field (Time-based Blind)

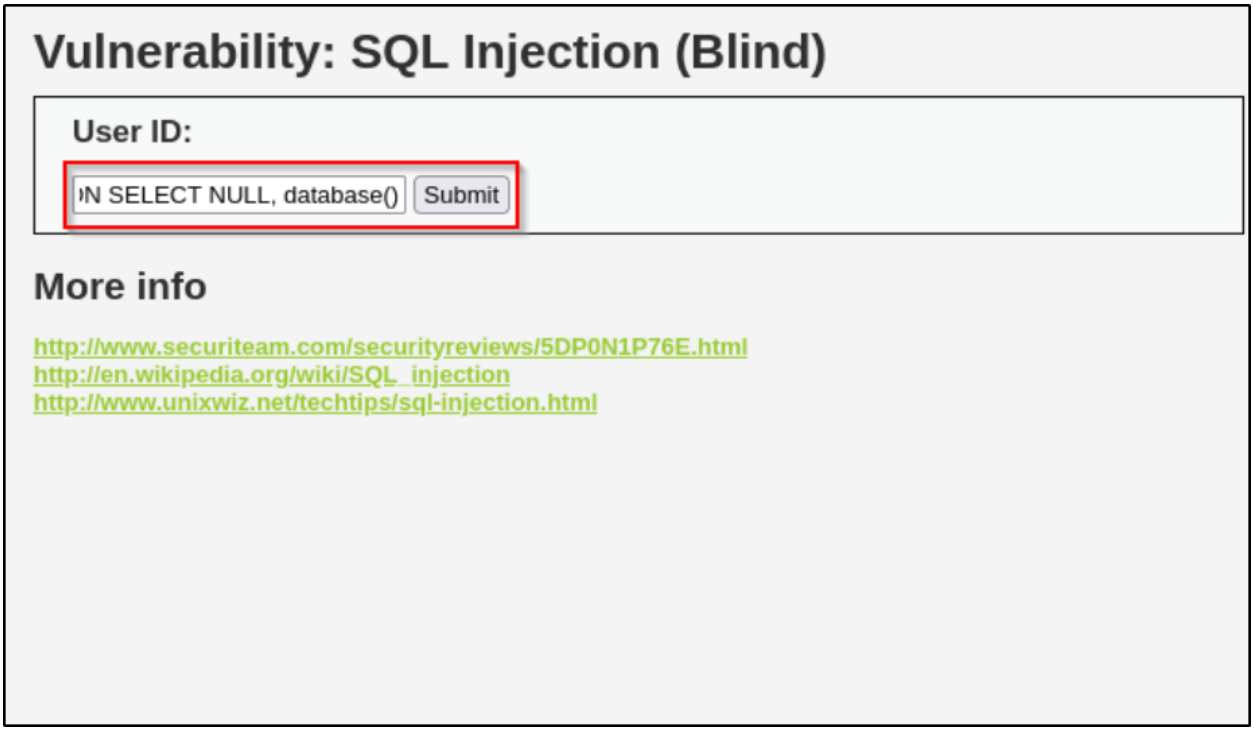


Figure 16: SQL injection payload entered into the "User ID" field (Union-based)

Copyright © RAAD Security (raad-security.com)

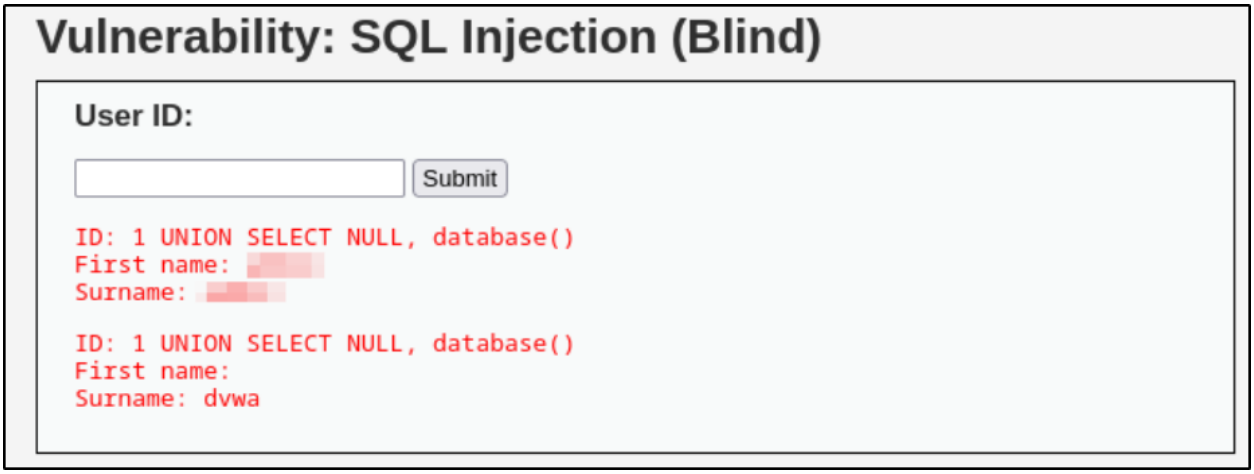


Figure 17: The query combines results with legitimate rows to reveal database information (e.g., database name)

Remediation

- Use parameterized queries or prepared statements to avoid direct concatenation of user input in SQL statements.
- Validate all user inputs for type, length, and format at the point of entry.
- Apply the principle of least privilege: restrict the application database account to only necessary privileges; remove DROP, FILE, SHUTDOWN, SUPER, etc.
- Secure password storage: use strong, salted hashing algorithms like bcrypt or argon2; avoid MD5 or weak hashes.
- Disable verbose SQL error messages in the user interface; log errors securely for auditing.
- Monitor and alert on unusual database activity or potential SQL injection attempts.
- Maintain secure backups and verify restoration procedures to mitigate potential data loss or destructive attacks.

Copyright © RAAD Security (raad-security.com)

Finding WAPT-005: Outdated / Unsupported Software Stack – Apache, PHP, mod_dav, Ubuntu (Critical)

Description:	The DVWA application is deployed on an outdated technology stack: Apache HTTP Server 2.2.8, Ubuntu 8.04, PHP 5.2.4, and mod_dav v2. These components are no longer supported and contain known vulnerabilities. Running unsupported software significantly increases the risk of remote exploits, including Remote Code Execution, file inclusion, Denial of Service, and information disclosure.
Risk:	Likelihood**:** High – Known exploits exist for all deployed components, making the environment highly susceptible to attack. Impact: Critical – Exploitation can result in full compromise of system confidentiality, integrity, and availability.
System:	https://19x.xx.xx.xx/dvwa/
Tools Used:	Wappalyzer
References:	https://fridge.ubuntu.com/2025/08/08/ubuntu-24-04-3-lts-released/ https://php.watch/versions/8.5 https://downloads.apache.org/httpd/Announcement2.4.html https://sabre.io/dav/

Evidence

Copyright © RAAD Security (raad-security.com)

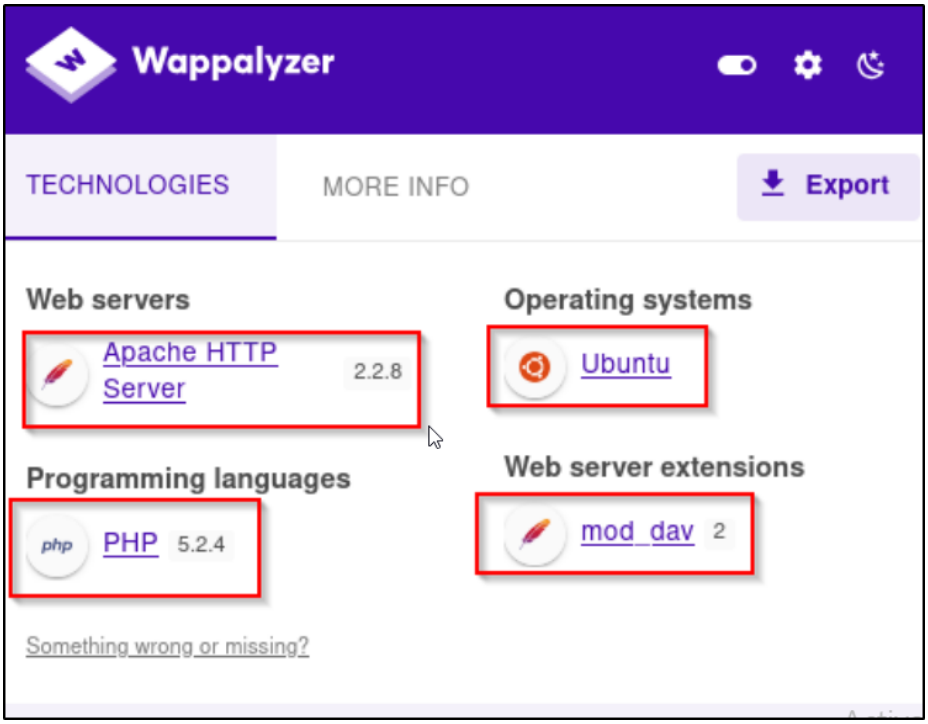


Figure 18: Outdated / Unpatched Software

Remediation

To mitigate the risks associated with outdated and unsupported software components in the DVWA application, it is strongly recommended to upgrade to the latest stable versions:

- Apache HTTP Server**:** Version 2.4.x (latest stable)
- PHP**:** Version 8.x (latest stable)
- Ubuntu**:** Latest LTS release (e.g., 22.04 or newer)
- mod_dav**:** Disable legacy mod_dav; use Apache 2.4's built-in mod_dav or a secure alternative such as SabreDAV 4.4.x

Upgrading ensures the application benefits from the latest security patches, performance improvements, and ongoing support, reducing the risk of exploitation due to known vulnerabilities.

Copyright © RAAD Security (raad-security.com)

Finding WAPT-006: Cross-Site Scripting (XSS) (High)

Description:	The DVWA application is vulnerable to Cross-Site Scripting (XSS) due to insufficient input validation and output encoding. Both reflected and stored variants exist: Reflected XSS: Malicious scripts are reflected immediately in HTTP responses, requiring user interaction (e.g., clicking a crafted link). Stored XSS: Malicious scripts are stored in the database and executed automatically in any user's browser when visiting the affected page. Exploitation may lead to session hijacking, data theft, defacement, or other malicious activity
Risk:	Likelihood: High – Exploitation can be carried out using simple input fields or crafted URLs. Impact: High – Both XSS types compromise user session integrity and confidentiality, with Stored XSS affecting multiple users automatically.
System:	https://19x.xx.xx.xx/dvwa/vulnerabilities/xss_r/ https://19x.xx.xx.xx/dvwa/vulnerabilities/xss_s/
Tools Used:	-
References:	https://owasp.org/www-community/attacks/xss/

Evidence

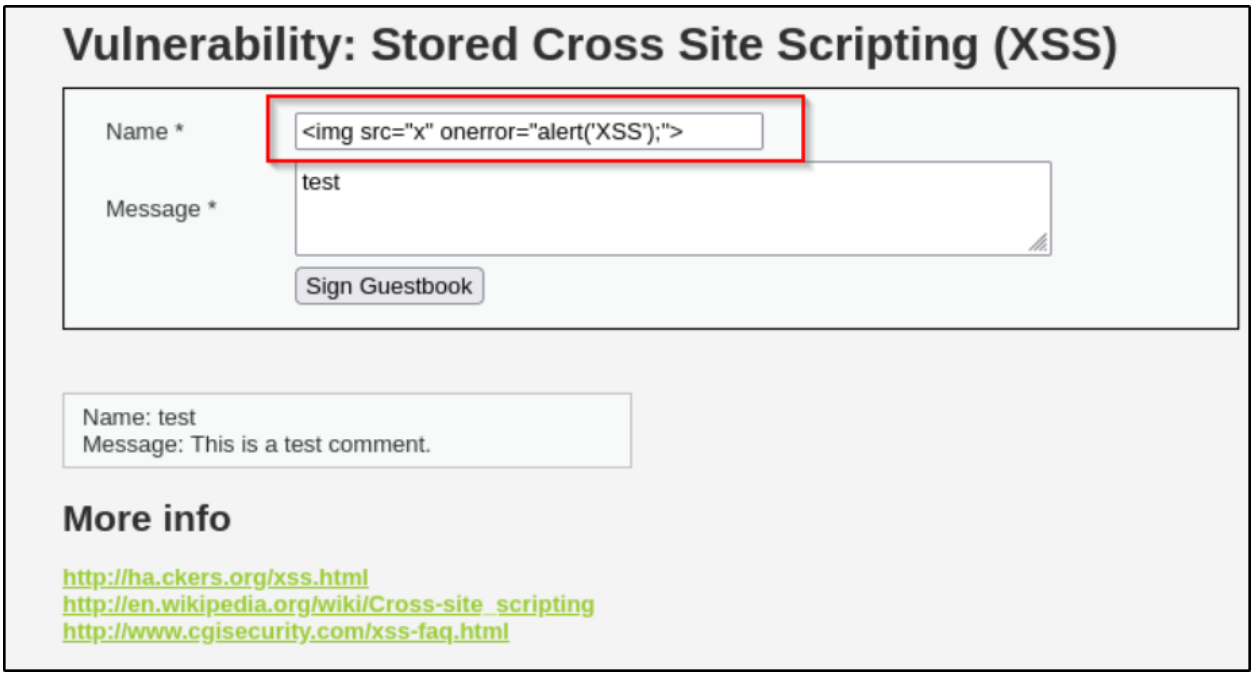


Figure 19: Malicious payload into the "Name" field allows the script to be stored in the database

Copyright © RAAD Security (raad-security.com)

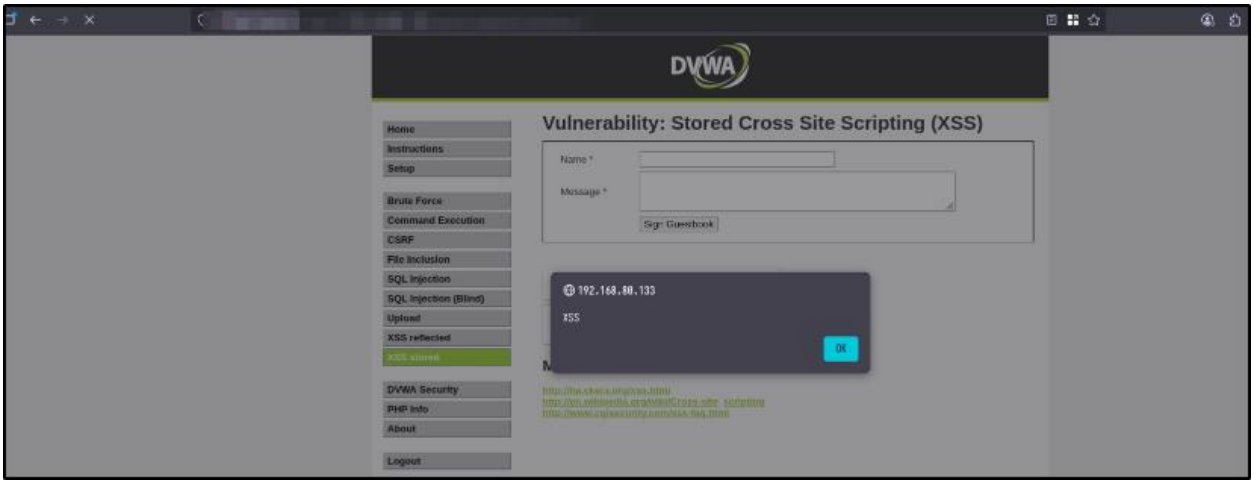


Figure 19: Malicious payload is triggered when the page is viewed

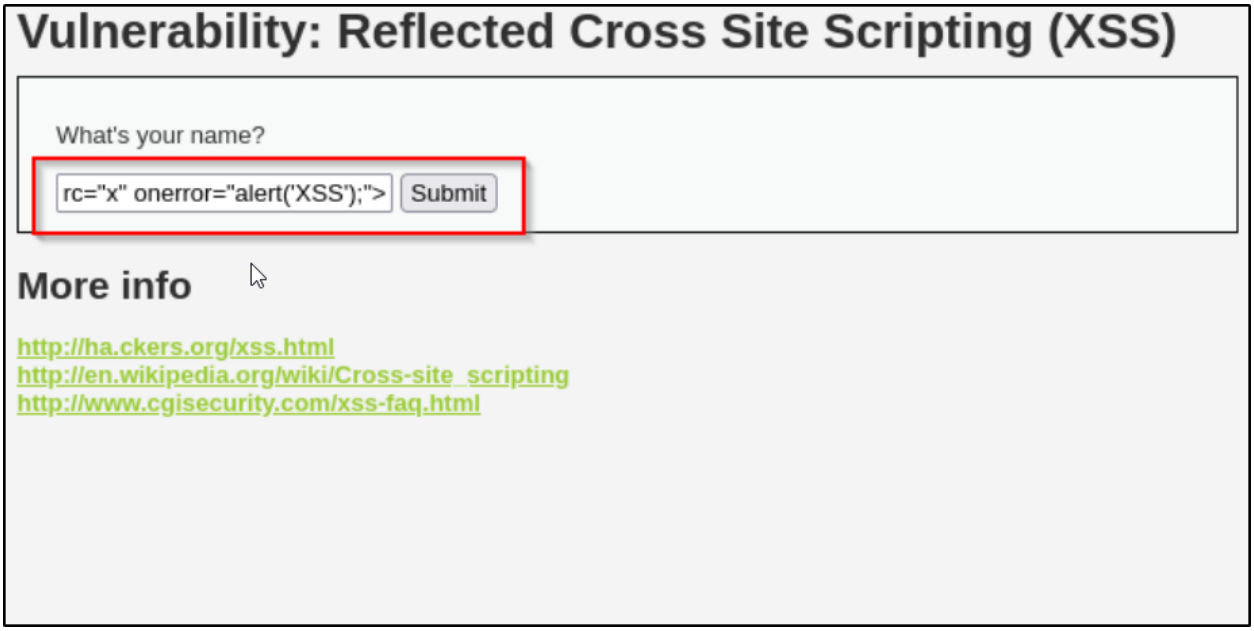


Figure 20: Malicious payload placed into the "What's your name?" field

Copyright © RAAD Security (raad-security.com)

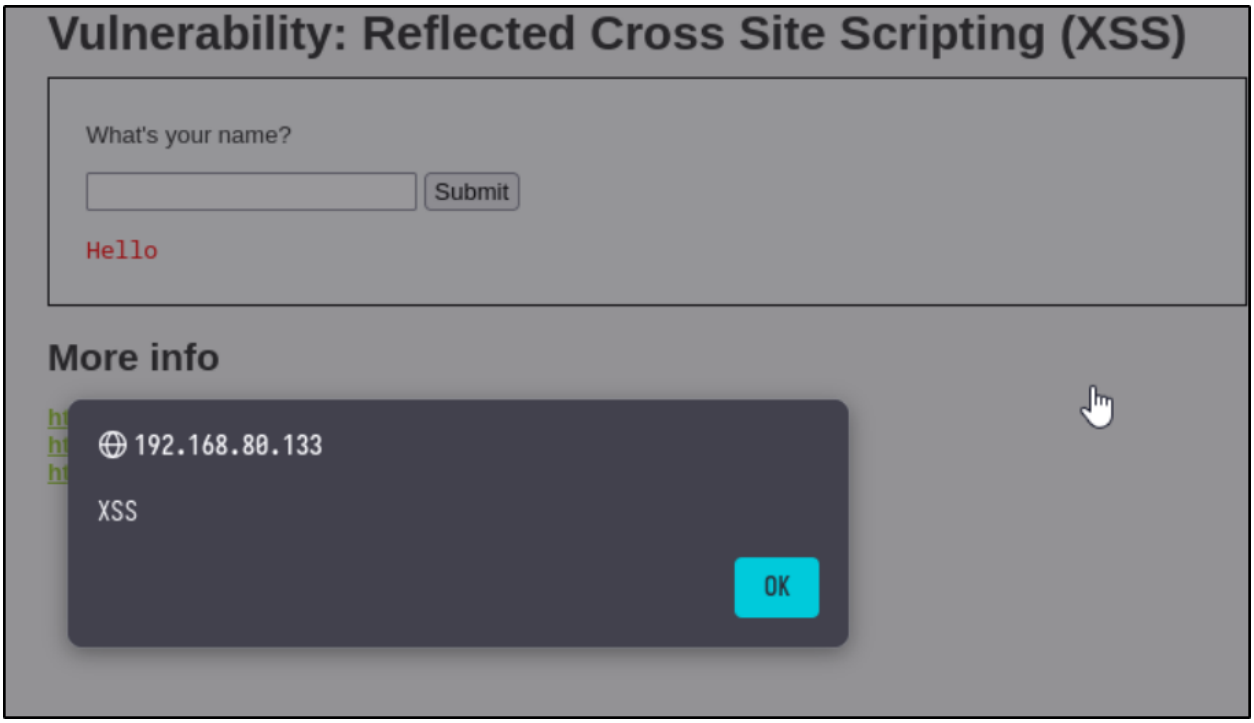


Figure 21: Malicious payload triggered when the user clicks the "Submit" button

Remediation

During the assessment, stored XSS vulnerabilities were identified in multiple locations,

indicating a persistent issue throughout the site. RAADSEC recommends the following actions:

- Filter input on arrival**:** Validate and sanitize user input based on expected format and type.
- Encode data on output: Apply context-aware output encoding (HTML, JavaScript, URL, etc.) before rendering user-controlled data.
- Use appropriate security headers**:** Enforce headers such as Content-Type and X-Content-Type-Options to ensure correct browser interpretation.
- Implement Content Security Policy **(**CSP**):** Deploy CSP as a defense-in-depth control to limit script execution sources.
- Monitoring & logging**:** Detect and alert on suspicious script injection attempts.

Copyright © RAAD Security (raad-security.com)

Finding WAPT-007: File Inclusion (LFI/RFI) Due to Improper Input Validation (High)

Description:	The DVWA application allows user-controlled input to influence file inclusion routines without proper validation, resulting in a file inclusion vulnerability. An attacker may exploit this issue to read sensitive local files (Local File Inclusion – LFI) or, if remote file inclusion is enabled, execute attacker-controlled code (Remote File Inclusion – RFI). Successful exploitation could lead to sensitive data disclosure or remote code execution.
Risk:	Likelihood: High – Exploitation requires only manipulation of user-controlled parameters and can be performed with minimal effort. Impact: High – Attackers may access sensitive files and potentially execute arbitrary code, compromising confidentiality and system integrity.
System:	https://19x.xx.xx.xx/dvwa/vulnerabilities/fi/?page=include.php
Tools Used:	Burp Suite
References:	https://owasp.org/www-community/attacks/Path_Traversal

Evidence

Figure 22: File inclusion payload entered into the "page" parameter

Figure 23: Partial `/etc/passwd` file retrieved via the vulnerable endpoint

Copyright © RAAD Security (raad-security.com)

Remediation

RAADSEC identified insufficient validation in file inclusion functionality, which may allow attackers to read sensitive files or potentially execute arbitrary code. To mitigate this risk, RAADSEC recommends the following:

- Implement strict allowlisting**:** Only permit predefined, trusted file names or identifiers. Avoid accepting arbitrary file paths from user input.
- Avoid dynamic file inclusion**:** Do not directly concatenate user-controlled input into file paths. Use internal mappings (e.g., ID-to-file mapping) instead of raw file names.
- Canonicalize and validate paths**:** Normalize file paths before processing and reject any input containing directory traversal sequences (e.g., ../).
- Disable remote file inclusion**:** Ensure remote URL inclusion is disabled at the application and server configuration levels unless explicitly required.
- Apply least privilege**:** Run the web server under restricted accounts and limit access to sensitive system directories.
- Monitoring and logging**:** Log file access attempts and implement alerting for suspicious patterns such as repeated traversal attempts.
- Implementing these controls will significantly reduce the risk of file inclusion exploitation and strengthen overall application security posture.

Copyright © RAAD Security (raad-security.com)

Finding WAPT-008: Insufficient CSRF Protection (Medium)

Description:	The DVWA application lacks proper Cross-Site Request Forgery (CSRF) protections on certain state-changing functionalities. The application does not adequately verify that requests originate from the authenticated user, allowing attackers to trick logged-in users into performing unintended actions. Exploitation may enable unauthorized changes to account settings or other sensitive operations without the victim's knowledge or consent. This attack can be carried out remotely through social engineering techniques, requiring only that the victim visit a crafted malicious page
Risk:	Likelihood: Medium – Exploitation requires the victim to be authenticated and interact with a malicious link or page, but the attack can be executed remotely with minimal technical complexity. Impact: Medium – Unauthorized actions may be performed on behalf of authenticated users, potentially affecting account integrity and application data.
System:	https://19x.xx.xx.xx/dvwa/vulnerabilities/csrf/
Tools Used:	-
References:	https://owasp.org/www-community/attacks/csrf

Evidence

```
<div class="vulnerable_code_area">

  <h3>
    Change your admin password:
  </h3>
  <br>
  <form action="#" method="GET">
        New password:<br>
    <input type="password" AUTOCOMPLETE="off" name="password_new">
    <br>
    Confirm new password: <br>
    <input type="password" AUTOCOMPLETE="off" name="password_conf">
    <br>
    <input type="submit" value="Change" name="Change">
  </form>


</div>
```

Figure 24: Password change form lacking CSRF protection

Copyright © RAAD Security (raad-security.com)

```
<form action=[redacted]/dvwa/vulnerabilities/csrf/?
password_new=12345678&password_conf=12345678&Change=Change#" method="GET">    New password:<br>
 <input type="password" AUTOCOMPLETE="off" name="password_new"><br>
 Confirm new password: <br>
 <input type="password" AUTOCOMPLETE="off" name="password_conf">
 <br>
 <input type="submit" value="Change" name="Change">
 </form>
```

Figure 25: Malicious CSRF form crafted to change the password of an authenticated user without their consent

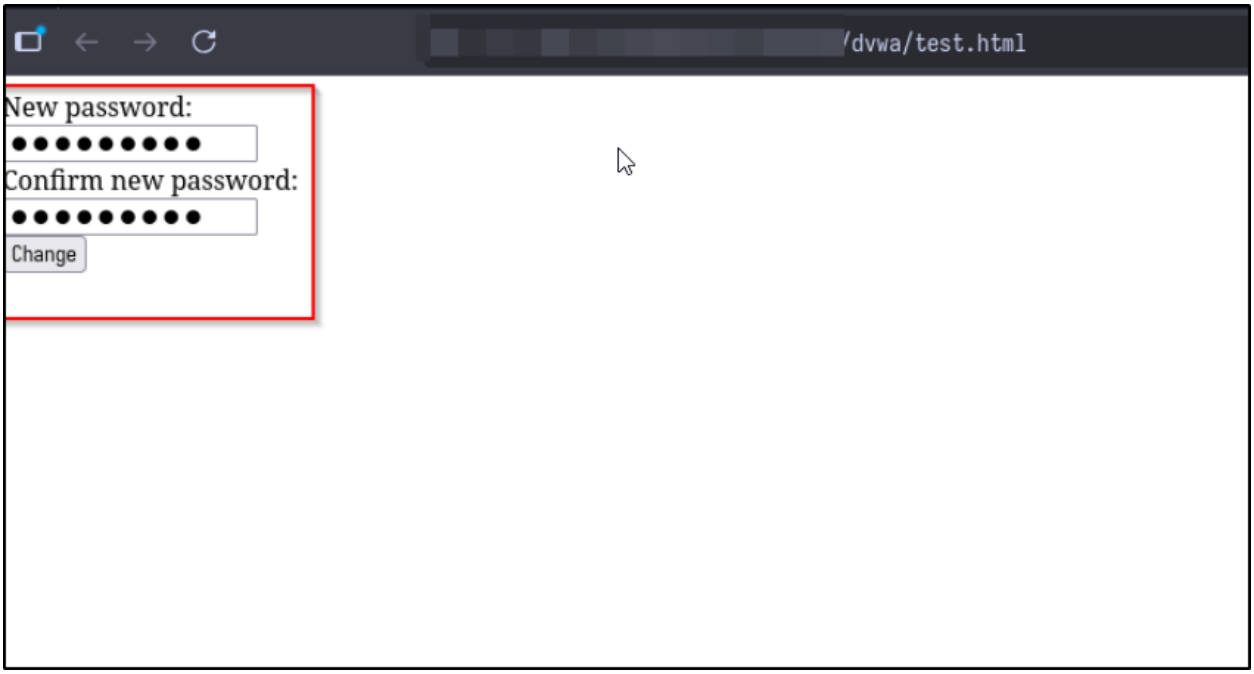


Figure 26: CSRF attack page opened in the browser with the new password filled

Copyright © RAAD Security (raad-security.com)

Figure 27: CSRF attack successfully executed, resulting in the password being changed without the user's consent

Remediation

RAADSEC recommends implementing the following controls:

- Implement anti-CSRF tokens**:** Include a unique, unpredictable token in all state-changing requests and validate it server-side.
- Enforce SameSite cookie attributes**:** Set session cookies to SameSite=Strict or SameSite=Lax to reduce cross-site request risks.
- Validate request origin**:** Verify the Origin and/or Referer headers for sensitive operations to ensure requests originate from trusted domains.
- Restrict state-changing methods**:** Avoid using GET for operations that modify data. Use POST requests protected with CSRF tokens.
- Re-authentication for critical actions**:** Require additional verification for sensitive account modifications.
- Implementing these measures will significantly reduce the risk of CSRF exploitation and strengthen overall session security.

Copyright © RAAD Security (raad-security.com)

Additional Scans and Reports

RAADSEC provides all clients with all report information gathered during testing. This includes Nessus files and full vulnerability scans in detailed formats. These reports contain raw vulnerability scans and additional vulnerabilities not exploited by RAAD Security.

The reports identify hygiene issues needing attention but are less likely to lead to a breach, i.e. defense-in-depth opportunities. For more information, please see the documents in your shared drive folder labeled “Additional Scans and Reports”.

Copyright © RAAD Security (raad-security.com)

RAAD

Security

Last Page

DVWA CORP

BUSINESS CONFIDENTIAL

Copyright © RAAD Security (raad-security.com)